

MeetText - Automated Business Meetings Podcast Transcription and Topic Segmentation

ABSTRACT

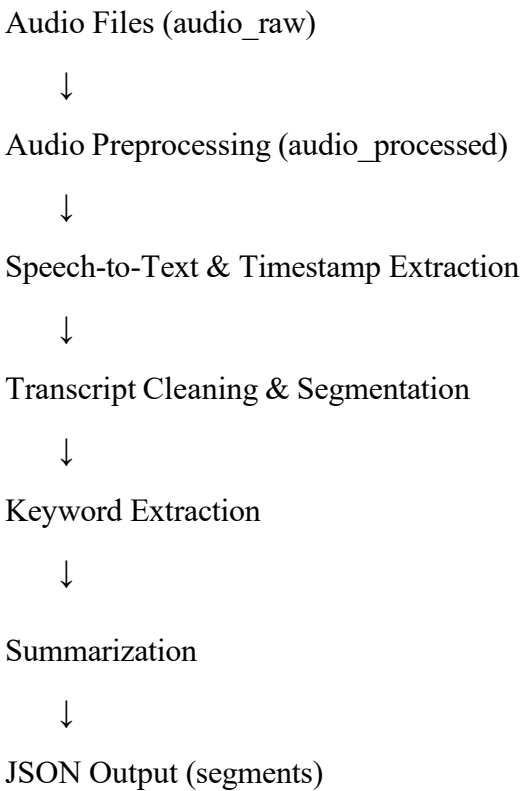
This project presents an automated system for analyzing meeting recordings by generating timestamp-aligned transcript segments and concise summaries. The system processes audio and transcript data to clean, segment, and align text with corresponding audio timestamps. It further extracts key discussion topics and produces structured summaries. The final output is stored in a machine-readable format, enabling efficient retrieval and analysis of long meeting recordings while significantly reducing manual review effort.

APPLICATION

The application is designed to assist users in efficiently reviewing long meeting or podcast recordings. Instead of listening to entire audio files, users receive timestamped transcript segments, extracted keywords, and concise summaries. The modular design allows easy extension and integration into other systems such as dashboards or analytics platforms.

SYSTEM ARCHITECTURE AND FLOW

System Workflow



This flow ensures structured and scalable processing of meeting data.

PROJECT DIRECTORY STRUCTURE

project/

```
|
|
|— audio_raw/      # Original audio input files
|— audio_processed/ # Cleaned and normalized audio
|— transcripts/    # Transcript text files
|— segments/      # Timestamped JSON outputs
|— notebooks/     # Experiments and analysis
|
|— src/
|   |— preprocessing.py # Audio preprocessing logic
|   |— transcription.py # Speech-to-text and timestamps
|   |— segmentation.py  # Transcript segmentation
|   |— keyword_extraction.py # Keyword identification
|   |— summarization.py  # Summary generation
|   |— ui_app.py         # User interface layer
|
|— docs/              # Documentation files
|— tests/             # Test cases
|— README.md         # Project overview
|— requirements.txt   # Dependencies
|— LICENSE           # License information
```

SETUP

The setup phase outlines the technical environment required to implement and execute the automated meeting transcription and summarization system. A properly configured environment ensures accurate audio processing, efficient transcription, and reliable output generation. This phase focuses on preparing both hardware and software components necessary for seamless system operation.

Hardware Requirements

- A computer system with **at least 8 GB RAM**, which is essential for handling audio processing and natural language tasks efficiently
- **Intel i5 / AMD Ryzen 5 processor or higher**, recommended for faster transcription and segmentation
- Adequate storage space to accommodate raw audio files, processed audio, transcripts, and structured output files

Software Requirements

- **Python 3.x**, used for implementing all core modules

- **Windows or Linux operating system** to support the required libraries
- Stable internet connectivity for installing dependencies and downloading speech models

Installation Procedure

1. Install Python and verify its availability using the command:
2. `python --version`
3. Create and activate a virtual environment to maintain dependency isolation.
4. Install all required libraries using the requirements.txt file provided with the project.
5. Verify that all project directories are correctly created before execution.

Upon successful completion of these steps, the system is ready for processing meeting recordings.

CORE METHODOLOGY

This section explains the **end-to-end logic** implemented in the project.

Step 1: Audio Preprocessing

Module: preprocessing.py

Raw audio files are loaded from the audio_raw directory.

Noise reduction and normalization techniques are applied.

Cleaned audio files are saved in audio_processed for accurate transcription.

Step 2: Speech-to-Text and Timestamp Extraction

Module: transcription.py

Processed audio files are converted into text.

Start and end timestamps are extracted for spoken segments.

Generated transcripts are stored in the transcript's directory.

Step 3: Transcript Cleaning

Transcript text is cleaned by removing empty lines and unwanted symbols.

Ensures high-quality input for segmentation.

Step 4: Sentence Tokenization and Segmentation

Cleaned transcripts are split into sentences.

Sentences are grouped into fixed-size segments to represent meaningful discussion blocks.

Step 5: Timestamp Alignment

Transcript segments are aligned with audio timestamps.

Each segment is assigned a corresponding start time from the audio.

Step 6: Keyword Extraction

Important keywords are extracted from the transcript.

Helps identify key discussion topics within the meeting.

Step 7: Summary Generation

Module: summarization.py

Segmented transcript text is summarized into a concise form.

The summary captures the main ideas discussed during the meeting.

Step 8: Output Generation

Module: summarization.py

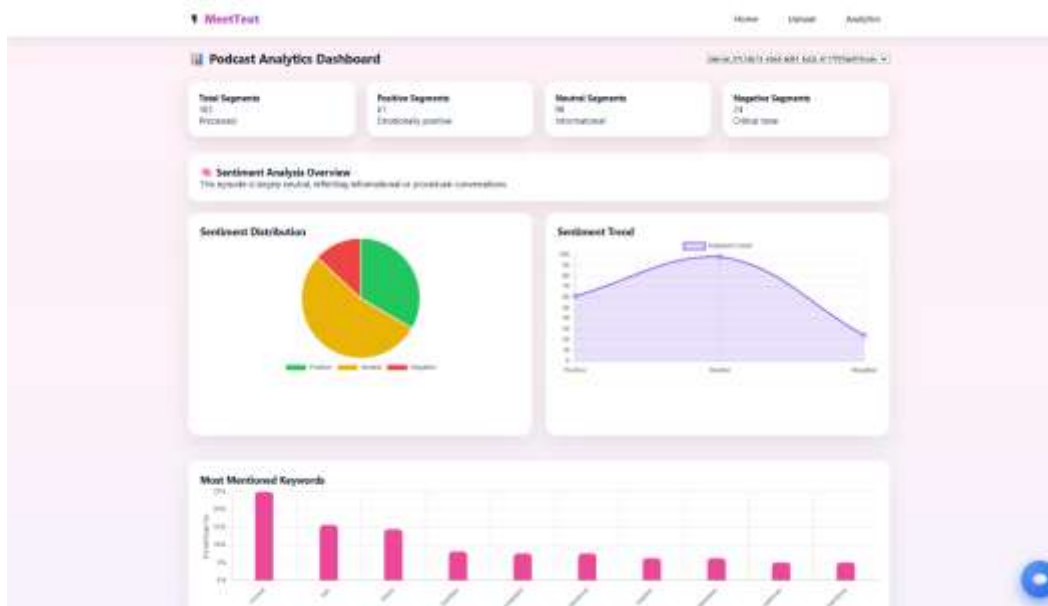
All results are stored in structured JSON files:

1. Filename
2. Timestamped segments
3. Extracted keywords
4. Generated summary

USAGE

The system is designed to follow a simple yet effective execution workflow. Users begin by placing meeting or podcast audio recordings in the **audio raw** directory. The preprocessing module then removes background noise and normalizes audio levels to enhance transcription accuracy. Cleaned audio is passed to the speech-to-text module, which converts spoken content into text while capturing accurate timestamps.

The generated transcript is further cleaned and segmented into meaningful discussion blocks. Keyword extraction identifies important topics, and the summarization module generates concise summaries that reflect the core ideas discussed during the meeting. All outputs are saved in structured **JSON format**, enabling easy review, storage, and future integration.



TROUBLESHOOTING

During the execution of the automated meeting transcription and summarization system, users may encounter certain technical or operational issues. The following points describe common problems along with their possible causes and solutions:

1. Low Transcription Accuracy
 - Occurs due to background noise, unclear speech, or low-quality microphones.
 - Can be improved by using noise-free recordings and high-quality audio input devices.
2. Unsupported Audio Format
 - The system may fail to process audio files in unsupported formats.
 - Converting audio files to standard formats such as .wav or .mp3 resolves this issue.
3. Empty or Missing Transcript Files
 - This may occur if the audio file contains silence or very low volume.
 - Ensuring proper audio levels before processing helps avoid this problem.
4. Incorrect or Misaligned Timestamps
 - Overlapping speakers or rapid speech transitions can cause timestamp inaccuracies.
 - Clear pauses between speakers improve timestamp alignment.

LIMITATIONS AND CHALLENGES

Although the system delivers efficient and accurate results, it has certain limitations and challenges that affect its performance under specific conditions:

1. Dependence on Audio Quality

- The system's accuracy is highly dependent on clear and noise-free audio recordings.

2. Overlapping Speaker Problem

- When multiple participants speak simultaneously, segmentation and transcription accuracy decreases.

3. Language Dependency

- The system is primarily optimized for English language audio and may perform poorly with other languages.

4. No Real-Time Processing

- The system processes pre-recorded audio files and does not support live or real-time transcription.

5. High Computational Requirement

- Processing long or high-quality audio files requires significant CPU and memory resources.

FUTURE SCOPE

The project provides a strong foundation for advanced development. Future improvements may include real-time transcription for live meetings, speaker dualization to identify participants, and multi-language support. Sentiment and emotion analysis could offer deeper insights into meeting dynamics. Cloud deployment would improve scalability, while dashboard integration would enhance data visualization and user interaction.

REFERENCES

1. OpenAI Whisper – Speech Recognition Models
 2. Python Software Foundation – Official Documentation
 3. Natural Language Toolkit (NLTK) Documentation
 4. KeyBERT Keyword Extraction Framework
 5. Flask Web Framework Documentation
 6. Research papers on automated meeting transcription and summarization
-